



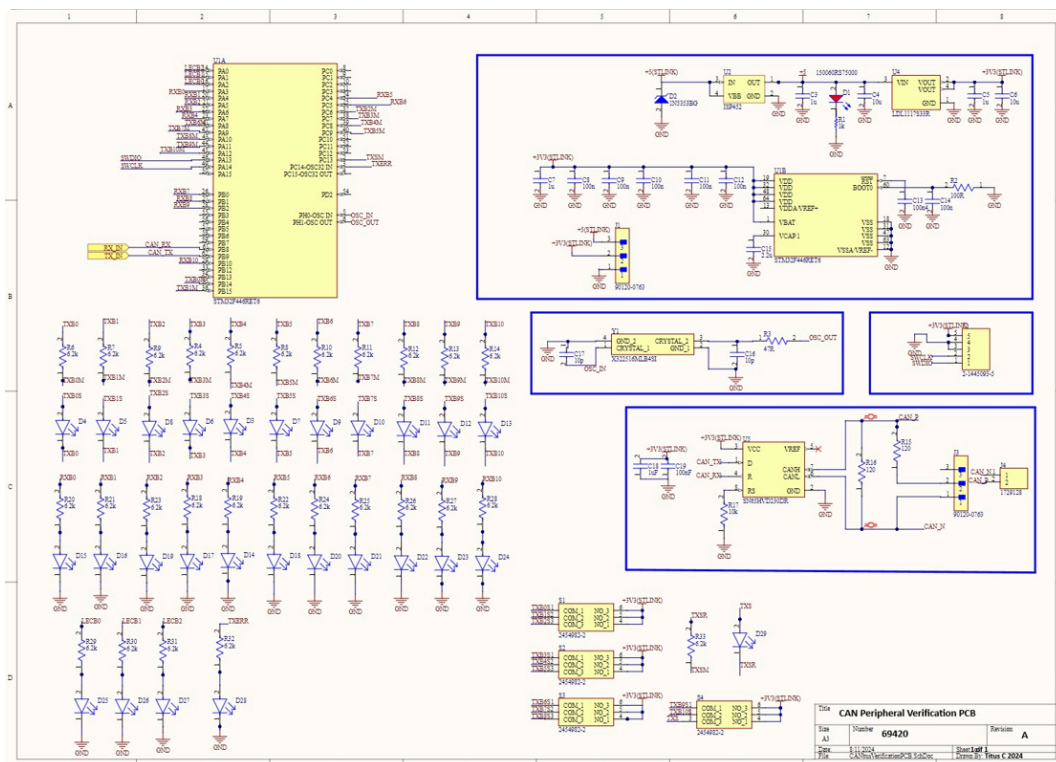
STM32 Upskilling Project - CANBus Verification PCB

Introduction

- Introduction
- Project Purpose
- Skills You'll Learn
- Project Features
- LED States
- Error Codes (LEC)
- Testing Process Overview
- Terminology
- Step 1: Transmission Testing
- Step 2: Reception Testing
- Setup and Preparation for Testing STM32F446RET6 PCBs
 - Transmission Setup in STM32CubeIDE
 - Reception Setup in STM32CubeIDE
- Starting the upskilling project

Welcome to the [CAN Peripheral Verification PCB](#) project. This document will guide you through the steps required to use the **CAN Peripheral Verification PCB** to test and verify the CAN2.0A functionality on your STM32-based custom PCBs. By working through this guide, you'll gain valuable hands-on experience with CAN communication, peripheral testing, and PCB bring-up, while learning to configure a simple CAN setup using **STM32CubeIDE**.

🔗 [GitHub - tituscharoensri/CAN_Peripheral_Verification_PCB: Easily Verify Transmission and Reception of CAN2.0A CANbus Peripheral for P](#)
[CB bring up & testing](#)



Project Purpose

The purpose of this project is to simplify and streamline the verification of CAN transmission and reception for custom PCBs. Instead of using multiple components like Nucleo boards, breakout boards, and jumper wires, this CAN Verification PCB (referred to as PCB_A) eliminates extra failure points and allows quick and reliable testing of CAN peripherals on your custom PCBs (referred to as PCB_B).

- It's crucial to understand your firmware's capabilities and gain experience with software configuration before working on PCB design. This will ensure you have the knowledge to optimize your hardware, making it easier to handle and allowing you to make informed hardware/software trade-offs or decisions when necessary.

Skills You'll Learn

- Hand Soldering
- The embedded systems sweatshop experience
- Basic CAN communication protocol (CAN2.0A)
- Setting up CAN peripheral functionality on STM32F446RET6
- Debugging CAN transmissions and receptions
- Using STM32CubeIDE to configure CAN peripherals
- Working with hardware interfaces (DIP switches, LEDs, jumper wires)
- Basic PCB bring-up and testing process
- Character development due to the insurmountable grief experienced when nothing works

Project Features

The **CAN Peripheral Verification PCB** has several useful features to assist you in testing CAN functionality:

1. **Tx ID (Transmission ID):** LED indicators to display the 11-bit CAN identifier being transmitted.
2. **Rx ID (Reception ID):** LED indicators to display the 11-bit CAN identifier that has been received.
3. **Continuous Transmission Control:** The 12th LED in the Tx ID row controls whether continuous transmission attempts are enabled or disabled, preventing bus-off states.
4. **Last Error Code (LEC) Display:** Displays the 3-bit error code from the CAN_ESR register (e.g., Stuff Error, Form Error, Acknowledgment Error, etc.).
5. **Tx Error Indicator:** Displays the occurrence of a transmission error.
6. **Optional CAN Bus Termination:** Use DIP switches to configure termination for the bus.
7. **Power Options:** Easily power the PCB through debugger or external power supply.
8. **Fixed Baud Rate (1MHz):** Pre-configured for 1 MHz CAN communication (changing baud rate requires reflashing firmware).

LED States

- **Solid:** 1/Enabled/Occurred
- **Off:** 0/Disabled/Non-event

Error Codes (LEC)

- 000: No Error
- 001: Stuff Error
- 010: Form Error
- 011: Acknowledgment Error
- 100: Bit Recessive Error
- 101: Bit Dominant Error
- 110: CRC Error
- 111: Set by Software

Testing Process Overview

- When you are doing initial board bring up, you can connect 2 “Can Peripheral verification PCBs” together and watch the IDs being sent and received through the LED display. You can then change hardware & software configurations and see how they effect your error codes & error registers.

Terminology

- **CAN Verification PCB (PCB_A)**: The verification PCB used to test CAN functionality.
- **User PCB (PCB_B)**: The custom STM32 PCB being tested.

Step 1: Transmission Testing

1. **Connect CANbus Lines:**

- Connect the CAN_H and CAN_L lines between PCB_A and PCB_B.

2. **Configure Termination:**

- Use DIP switches on PCB_A to enable or disable termination based on your setup.

3. **Set Up PCB_B:**

- Configure PCB_B to transmit CAN packets with a chosen identifier (e.g., **0x555** for standard CAN2.0A frames).

4. **Power On Both PCBs:**

- Ensure both PCBs are powered up (either via debugger or external power).

5. **Verify Reception on PCB_A:**

- Check the **Rx ID** LED indicators on PCB_A to confirm that the correct identifier was received (e.g., **0b10101010101**).

Step 2: Reception Testing

1. Connect CANbus Lines:

- Connect the CAN_H and CAN_L lines between PCB_A and PCB_B.

2. Configure Termination:

- Use DIP switches on PCB_A to configure the appropriate termination.

3. Set DIP Switches on PCB_A:

- Set the DIP switches on PCB_A to transmit CAN packets with the desired identifier (e.g., **0x555**).

4. Power On Both PCBs:

- Ensure both PCBs are powered up (either via debugger or external power).

5. Check Reception on PCB_B:

- Check the status LEDs or other indicators on PCB_B to confirm reception of the transmitted identifier (e.g., **0x555**).

Setup and Preparation for Testing STM32F446RET6 PCBs

In this section, we'll walk you through the process of configuring CAN communication for both transmission and reception using **STM32CubeIDE**.

Transmission Setup in STM32CubeIDE

1. **Open CubeMX in STM32CubeIDE.**
2. **Enable CAN1 Peripheral:**
 - In the **Peripherals** tab, select CAN1.
3. **Configure CAN TX Pin:**
 - Choose the appropriate GPIO pin for **CAN_Tx** from the available options (e.g., **PA12**).
4. **Configure Baud Rate:**
 - Set the baud rate to **1MHz** (or your desired rate).
5. **Write CAN Transmission Code:**
 - Use the HAL function `HAL_CAN_AddTxMessage()` to queue a CAN message for transmission.

Reception Setup in STM32CubeIDE

1. **Enable CAN1 Peripheral:**
 - In the **Peripherals** tab, select CAN1.
2. **Configure CAN RX Pin:**
 - Choose the appropriate GPIO pin for **CAN_Rx** from the available options (e.g., **PA11**).
3. **Set Up Reception Filters:**
 - Configure CAN filters to accept specific identifiers.
4. **Write CAN Reception Code:**
 - Use the HAL function `HAL_CAN_GetRxMessage()` to retrieve incoming CAN messages.

Starting the upskilling project

i This is **NOT** meant to be done individually!! We will only need about 2-4 of this PCB to be made, so the idea is to form a group, work on this together, especially in the Firmware and debugging phase. Create a Bitbucket repository and work on it together throughout the term.

1. Solder the PCB

- a. There are some practice PCBs for Soldering Upskilling if you are unfamiliar.

Component	Where to find it (Embedded Sys shelves)
STM32 Chip	STM32 Components carboard box
Resistors	Resistors Box
Capacitors	Capacitors box & Wurth Electronics Sample box
LEDs	Power Electronics Box
Connectors	PCB connectors box
Boot0 Switch	STM32 Components carboard box
Crystal Oscillator	STM32 Components carboard box
CAN Transceiver	STM32 Components carboard box
Dip Switches	CANNode section in shelves

2. Read through:

- a. [STM32 Minimum Viable Code](#) - Introduction to key features to get your project working (Clocks, CAN, NVIC)
- b. [CANBus & STM32](#) - Indepth tutorial on CAN protocol and how it works on STM32F446RET6
- c. Watch the [STM32 Tutorial - Embedded Systems 2024](#) Video (This goes through the **entire Design process**, you may want to skip to "Firmware", but watching the full length is recommended for understanding). **What this video contains:**
 - i. **Setting up a project on CUBEIDE**
 - ii. **Translating Pinout to Altium**
 - iii. **PCB design basics and theory (traces, stack up, etc.)**
 - iv. **Editing schematics and components in Altium**
 - v. **Routing PCBs in Altium (Constraints, design rules, error markers, hotkeys etc.)**
 - vi. **PCB design guidelines for critical components**
 - vii. **How to use polygons, editing polygons etc.**
 - viii. **Defining board shapes & finishing touches**
 - ix. **Design Rule Check (DRC) & how to fix DRC Errors**
 - x. **Creating Gerber files & Fabrication Outputs**
 - xi. **Firmware (Clock Configurations, GPIOs, CAN TX & RX)**

3. Begin the grind (Refer to spec above)

- a. Start by testing different sections of the project in isolation first e.g (verify LEDs work, then verify DIP switch bits can be read, then verify CAN peripherals TX & RX working, then begin integration while testing)
- b. Work in groups! This project is 100% designed to be worked on by multiple people. Reason being we don't have the budget to make multiple of these PCBs 🤖😞
- c. Refer to Confluence documentation & also you tube, datasheets, reference manuals, forums etc.
- d. The [The "My STM32 is not flashing" Checklist](#) will help if you are having trouble flashing your project

4. Done!

- a. You have just made a PCB that is extremely useful to redback development and will make project bring up and testing much much less stressful 😊